

Lifecycle Methods: Overview

- JUnit lifecycle annotations allow code to run **before or after test execution**
- They help **organize setup and cleanup logic** in a predictable manner
- Methods with **lifecycle annotations** are **not test methods**
- Useful for **preparing resources** and ensuring **consistent test behavior**

 **by Rodrigo Martins Pagliares**

Last update: 11/26/2025



@BeforeAll Setup

- Runs **once**, before *any* test method in the class
- Ideal for creating heavy or expensive resources (e.g., database creation)
- Method **must be static** unless special test instance modes are used
- Common for initializing shared state needed by all tests



@BeforeAll Setup

The screenshot shows an IDE with a project named 'CalculatorMavenProject'. The file 'CalculatorTest.java' is open, showing the following code:

```
10
11 @DisplayName("Test operations in Calculator class")
12 public class CalculatorTest {
13
14     // Template for naming test methods:
15     // test<method>_<condition>_<expectedResult>
16
17     @BeforeAll
18     static void beforeAll() {
19         System.out.println("Executing @BeforeAll method");
20     }
21
22     @DisplayName("Test 10/2 = 5")
23     @Test
24     public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
25
```

The Run window at the bottom shows the execution of 'CalculatorTest'. It indicates that 2 tests passed out of 2 tests total, taking 16 ms. The test methods listed are 'Test 10/2 = 5' (5 ms) and 'Test 113-13=100' (11 ms). The output shows the path to the Java runtime and the message 'Process finished with exit code 0'.

- Runs **once**, before *any* test method in the class
- You can choose any name you like for this method (for instance, `setup`)

@AfterAll Cleanup

Executes once

After *all* test methods finish

Release resources

Used to release shared resources (e.g., drop DB, shut down servers)

Must be static

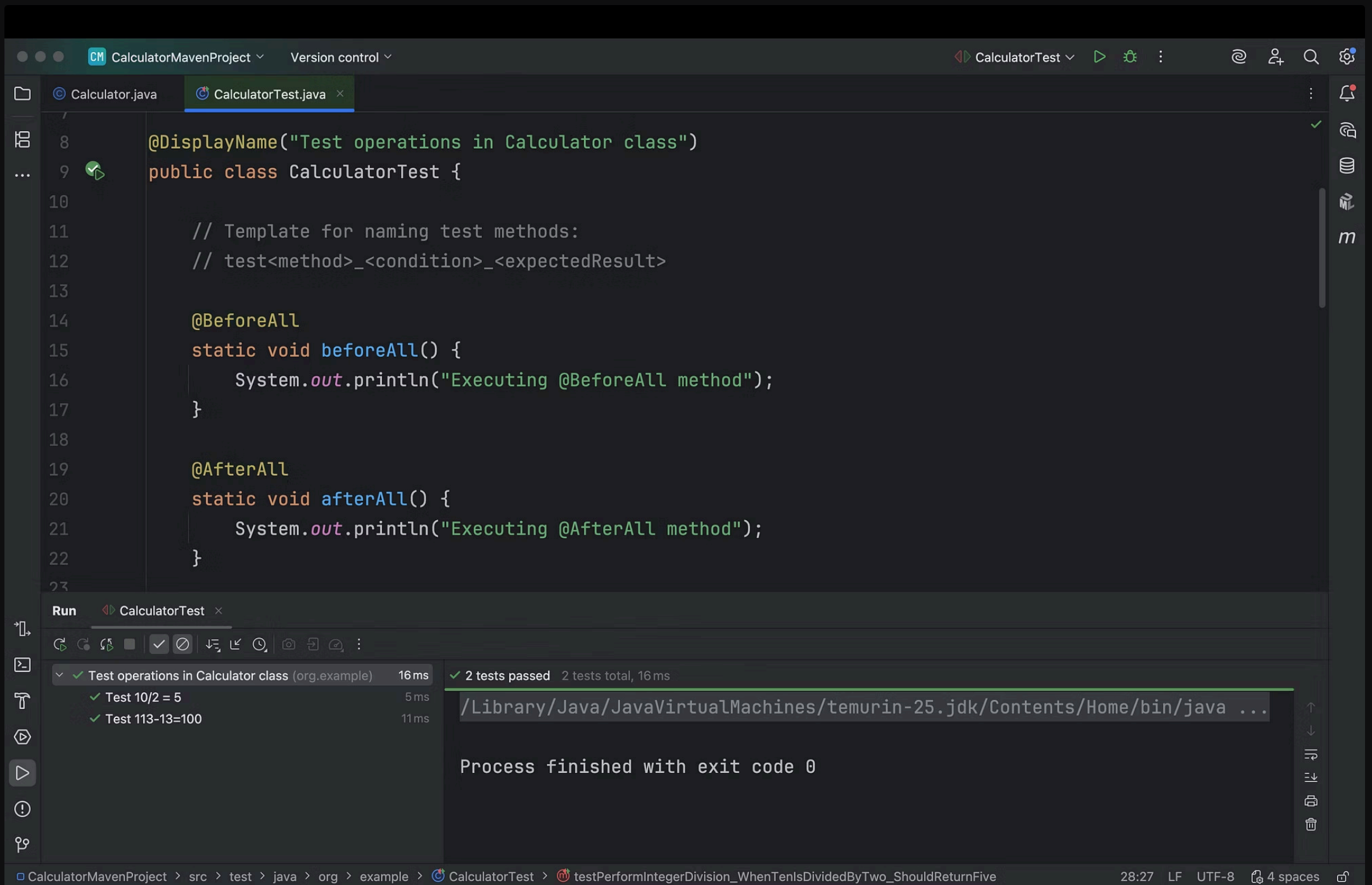
Also must be **static**

Clean exit

Ensures no resources are left open after test execution



@AfterAll Cleanup



```
8 @DisplayName("Test operations in Calculator class")
9 public class CalculatorTest {
10
11     // Template for naming test methods:
12     // test<method>_<condition>_<expectedResult>
13
14     @BeforeAll
15     static void beforeAll() {
16         System.out.println("Executing @BeforeAll method");
17     }
18
19     @AfterAll
20     static void afterAll() {
21         System.out.println("Executing @AfterAll method");
22     }
23 }
```

Run CalculatorTest

Test Name	Duration
Test operations in Calculator class (org.example)	16 ms
Test 10/2 = 5	5 ms
Test 113-13=100	11 ms

✓ 2 tests passed 2 tests total, 16 ms

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...

Process finished with exit code 0

Executes once After *all* test methods finish

- You can choose any name you like for this method (for instance, `cleanUp`)

A pig wearing a white chef's hat is standing in a kitchen. The pig is holding a knife and is about to cut a piece of food on a wooden cutting board. There are various kitchen items around, including a bowl of fruit, a bowl of vegetables, and a pot on the stove.

@BeforeEach Initialization

- Runs **before every** individual test method
- Useful for **creating fresh objects** used in tests
- **Eliminates duplicated initialization code** across test methods
- Helps ensure each **test begins** with a **clean state**

@BeforeEach Initialization

The screenshot displays an IDE window for a project named 'CalculatorMavenProject'. The file 'CalculatorTest.java' is open, showing the following code:

```
9 public class CalculatorTest {  
16     System.out.println("Executing @BeforeAll method");  
17 }  
18  
19 @AfterAll  
20 static void afterAll() {  
21     System.out.println("Executing @AfterAll method");  
22 }  
23  
24 @BeforeEach  
25 void beforeEachTestMethod() {  
26     System.out.println("Executing @BeforeEach method");  
27 }  
28  
29 @DisplayName("Test 10/2 = 5")  
30 @Test
```

Below the code editor, the 'Run' tab for 'CalculatorTest' is active. It shows a summary of test results:

Test Name	Duration
Test operations in Calculator class (org.example)	16 ms
Test 10/2 = 5	5 ms
Test 113-13=100	11 ms

The overall status is '2 tests passed' out of '2 tests total' in '16 ms'. The console output shows the command: `/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...` and the message: `Process finished with exit code 0`.

- Runs **before every** individual test method



@AfterEach Teardown

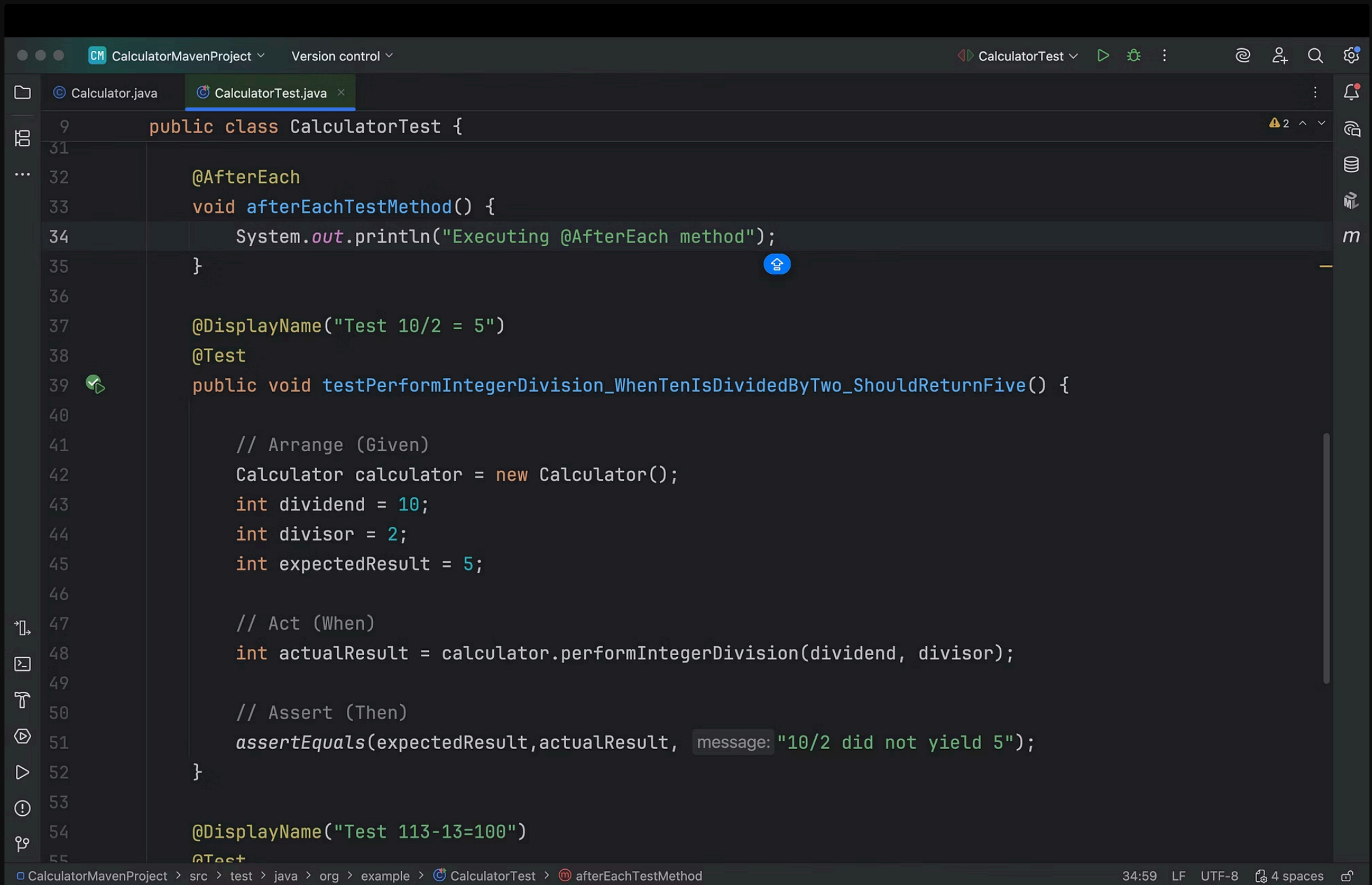
Runs **after every** individual test method

Used to clean temporary data or close lightweight resources

Ensures each test leaves the environment in a predictable state

Prevents interference between consecutive tests

@AfterEach Teardown



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The file 'CalculatorTest.java' is open, displaying the following Java code:

```
9      public class CalculatorTest {
31
32      @AfterEach
33      void afterEachTestMethod() {
34          System.out.println("Executing @AfterEach method");
35      }
36
37      @DisplayName("Test 10/2 = 5")
38      @Test
39      public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
40
41          // Arrange (Given)
42          Calculator calculator = new Calculator();
43          int dividend = 10;
44          int divisor = 2;
45          int expectedResult = 5;
46
47          // Act (When)
48          int actualResult = calculator.performIntegerDivision(dividend, divisor);
49
50          // Assert (Then)
51          assertEquals(expectedResult, actualResult, message: "10/2 did not yield 5");
52      }
53
54      @DisplayName("Test 113-13=100")
55      @Test
```

The IDE interface includes a sidebar with file explorer, a top toolbar with icons for running and debugging, and a bottom status bar showing the current file path: 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest > afterEachTestMethod'. The status bar also displays '34:59', 'LF', 'UTF-8', and '4 spaces'.

Runs **after every** individual test method

Example: Applying Lifecycle Methods



01

`@BeforeAll`: prints "Executing before all method" once

02

`@BeforeEach`: creates new Calculator object for each test

03

Test methods execute business logic and print identifying messages

04

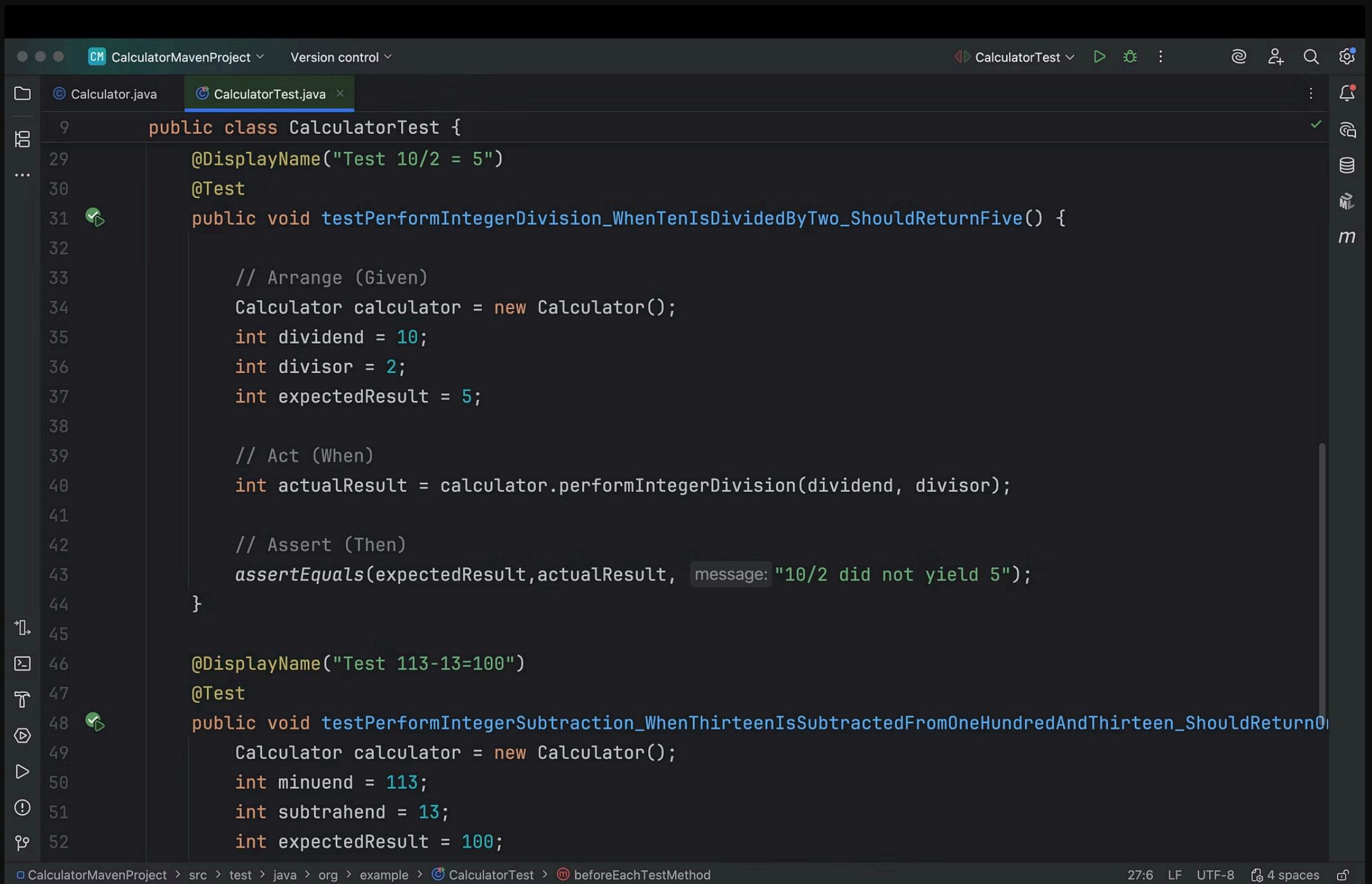
`@AfterEach` and `@AfterAll` print cleanup messages

05

Console output shows exact execution order: **All** → **Each** → **Test** → **Each** → ... → **All**

Example: Applying Lifecycle Methods(contd.)

@BeforeAll, @AfterAll, @BeforeEach, @AfterEach



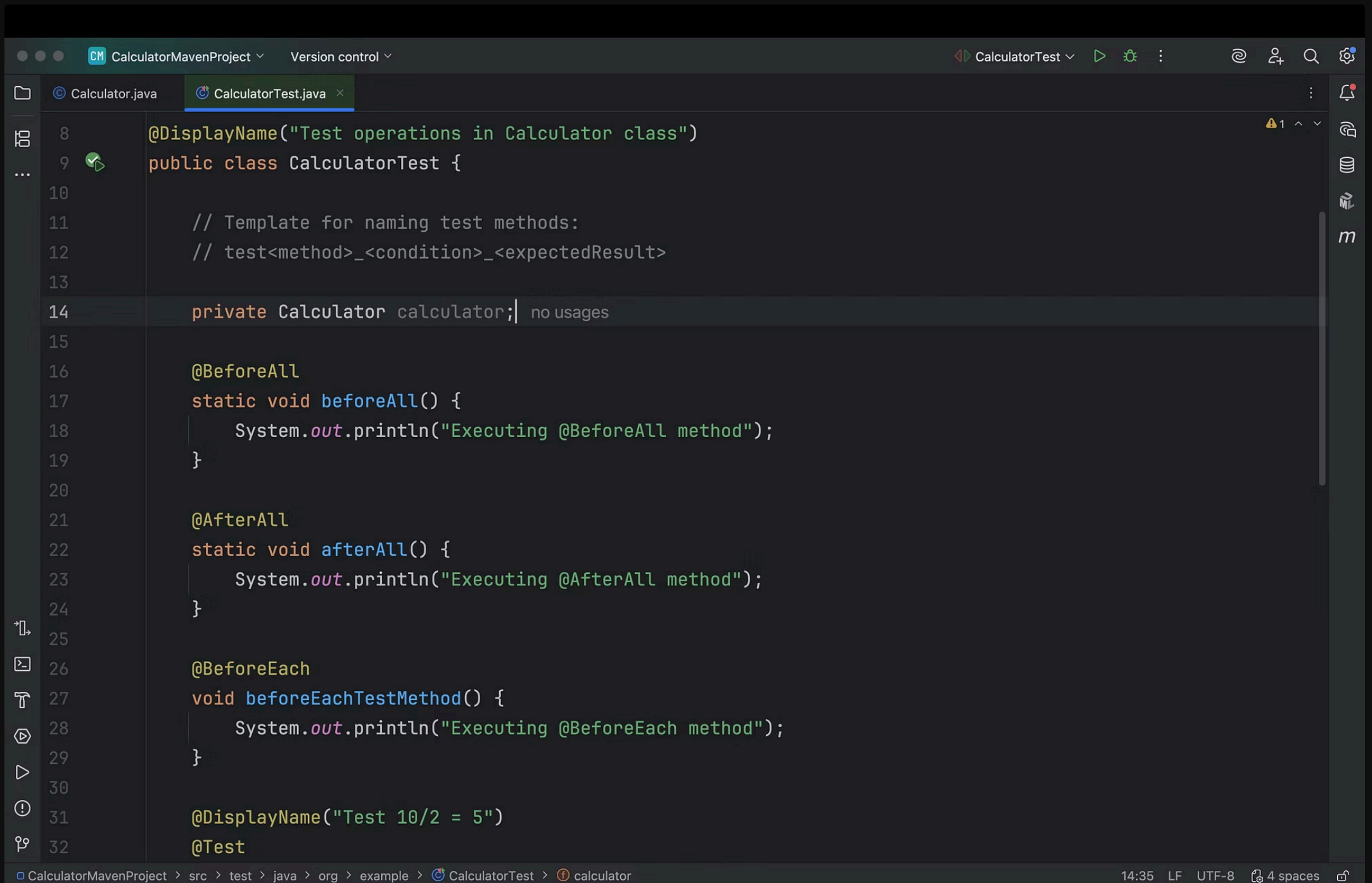
```
9      public class CalculatorTest {  
29         @DisplayName("Test 10/2 = 5")  
30         @Test  
31         public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {  
32  
33             // Arrange (Given)  
34             Calculator calculator = new Calculator();  
35             int dividend = 10;  
36             int divisor = 2;  
37             int expectedResult = 5;  
38  
39             // Act (When)  
40             int actualResult = calculator.performIntegerDivision(dividend, divisor);  
41  
42             // Assert (Then)  
43             assertEquals(expectedResult, actualResult, message: "10/2 did not yield 5");  
44         }  
45  
46         @DisplayName("Test 113-13=100")  
47         @Test  
48         public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturn0  
49             Calculator calculator = new Calculator();  
50             int minuend = 113;  
51             int subtrahend = 13;  
52             int expectedResult = 100;
```

Notice that the **Calculator instance** is created at **Lines 34** and **49**

Let's refactor the code in **5 steps** to leverage **@BeforeEach** method.

Example: Applying Lifecycle Methods(contd.)

@BeforeAll, @AfterAll, @BeforeEach, @AfterEach



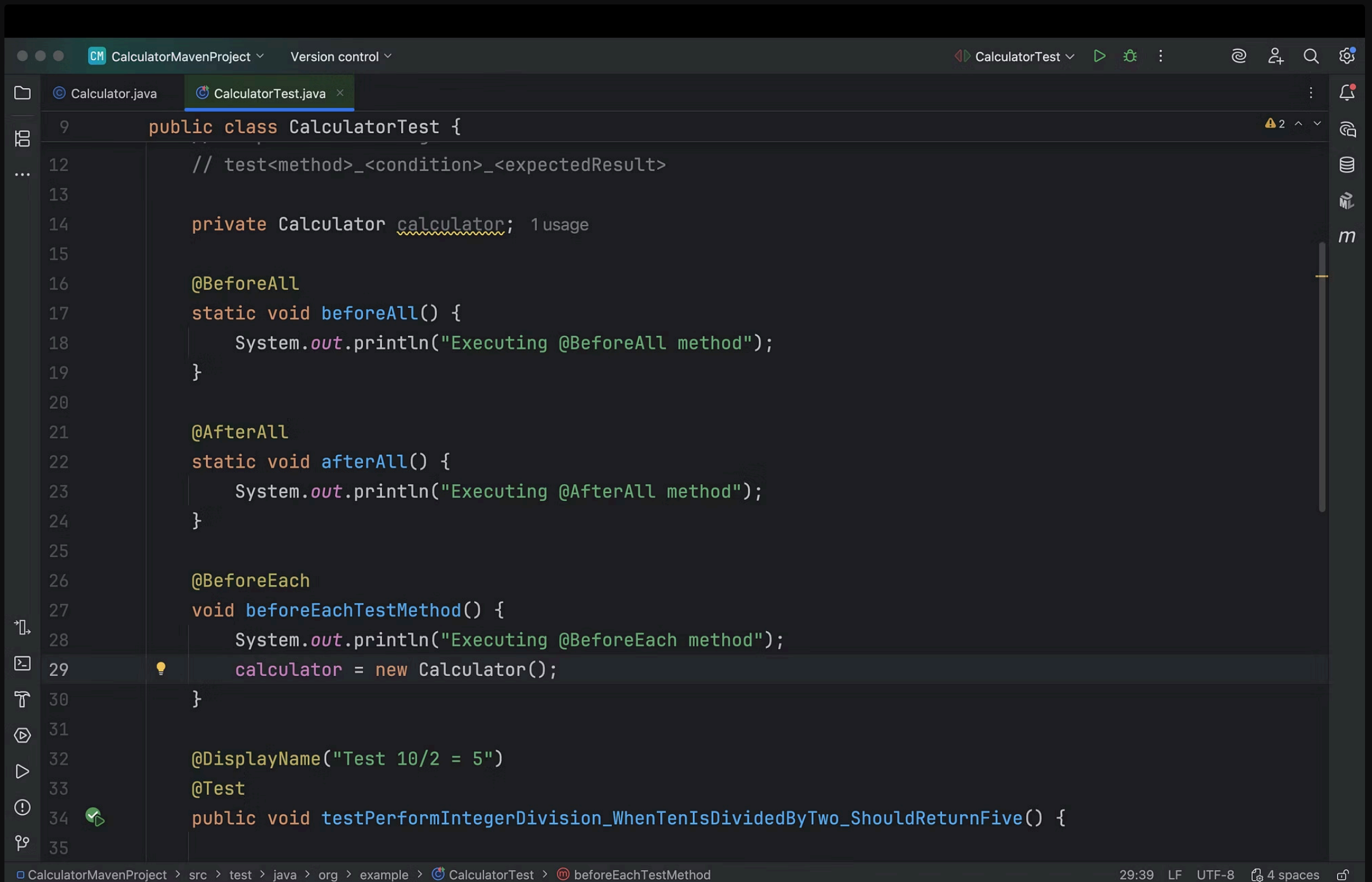
```
8  @DisplayName("Test operations in Calculator class")
9  public class CalculatorTest {
10
11      // Template for naming test methods:
12      // test<method>_<condition>_<expectedResult>
13
14      private Calculator calculator;| no usages
15
16      @BeforeAll
17      static void beforeAll() {
18          System.out.println("Executing @BeforeAll method");
19      }
20
21      @AfterAll
22      static void afterAll() {
23          System.out.println("Executing @AfterAll method");
24      }
25
26      @BeforeEach
27      void beforeEachTestMethod() {
28          System.out.println("Executing @BeforeEach method");
29      }
30
31      @DisplayName("Test 10/2 = 5")
32      @Test
```

First step: Create an instance variable (Line 14) with the instance to be created **BEFORE EACH** method.

```
private Calculator calculator;
```

Example: Applying Lifecycle Methods(contd.)

@BeforeAll, @AfterAll, @BeforeEach, @AfterEach

A screenshot of an IDE window showing the CalculatorTest.java file. The file contains the following code:

```
9      public class CalculatorTest {  
12         // test<method>_<condition>_<expectedResult>  
13  
14         private Calculator calculator; 1 usage  
15  
16         @BeforeAll  
17         static void beforeAll() {  
18             System.out.println("Executing @BeforeAll method");  
19         }  
20  
21         @AfterAll  
22         static void afterAll() {  
23             System.out.println("Executing @AfterAll method");  
24         }  
25  
26         @BeforeEach  
27         void beforeEachTestMethod() {  
28             System.out.println("Executing @BeforeEach method");  
29             calculator = new Calculator();  
30         }  
31  
32         @DisplayName("Test 10/2 = 5")  
33         @Test  
34         public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {  
35
```

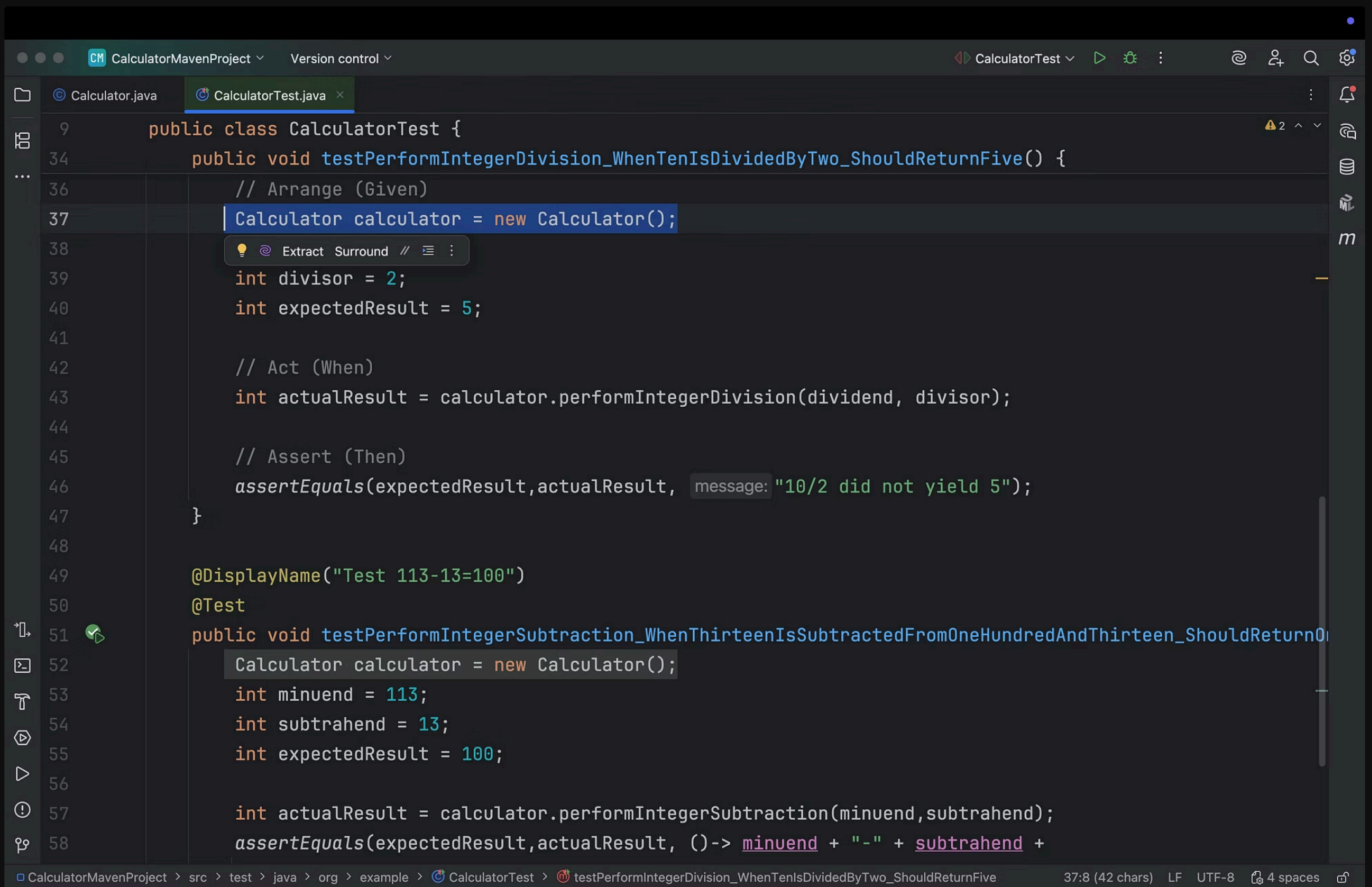
The IDE interface includes a top bar with project and version control information, a left sidebar with file explorer and search icons, and a right sidebar with various tool icons. The status bar at the bottom shows the current file path, line number (29:39), and encoding (UTF-8).

Second step: Instantiate the object in the @BeforeEach method.

```
calculator = new Calculator();
```

Example: Applying Lifecycle Methods(contd.)

@BeforeAll, @AfterAll, @BeforeEach, @AfterEach



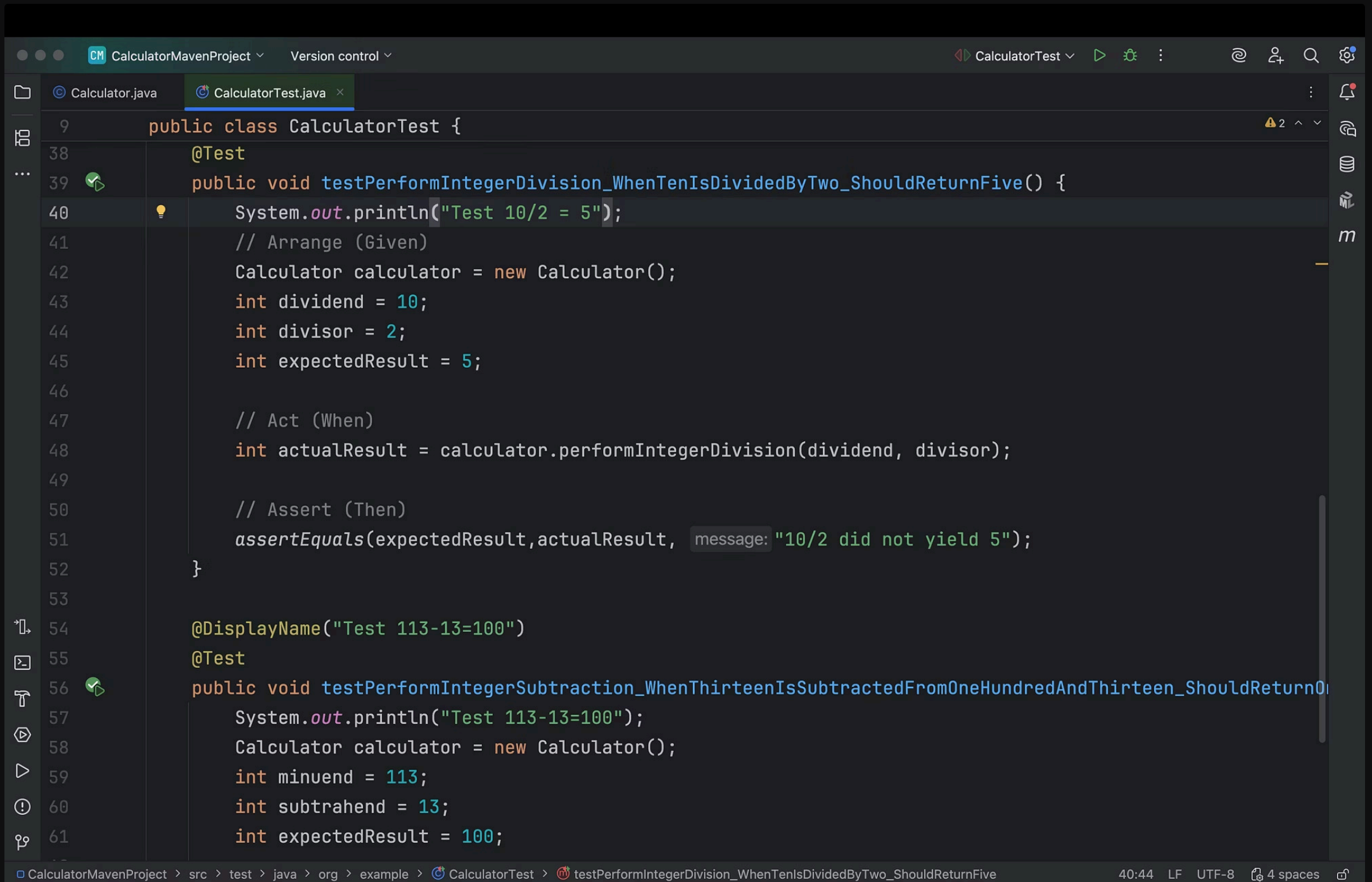
```
9      public class CalculatorTest {
34      public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
36          // Arrange (Given)
37          Calculator calculator = new Calculator();
38          // Extract Surround //
39          int divisor = 2;
40          int expectedResult = 5;
41
42          // Act (When)
43          int actualResult = calculator.performIntegerDivision(dividend, divisor);
44
45          // Assert (Then)
46          assertEquals(expectedResult, actualResult, message: "10/2 did not yield 5");
47      }
48
49      @DisplayName("Test 113-13=100")
50      @Test
51      public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturn0
52          Calculator calculator = new Calculator();
53          int minuend = 113;
54          int subtrahend = 13;
55          int expectedResult = 100;
56
57          int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
58          assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
```

Third step: Remove the **statement** that were used to create an **instance** of the **class** Calculator (now instantiated in the @BeforeEach method).

Calculator calculator = new Calculator(); // REMOVE THESE LINES!

Example: Applying Lifecycle Methods(contd.)

@BeforeAll, @AfterAll, @BeforeEach, @AfterEach

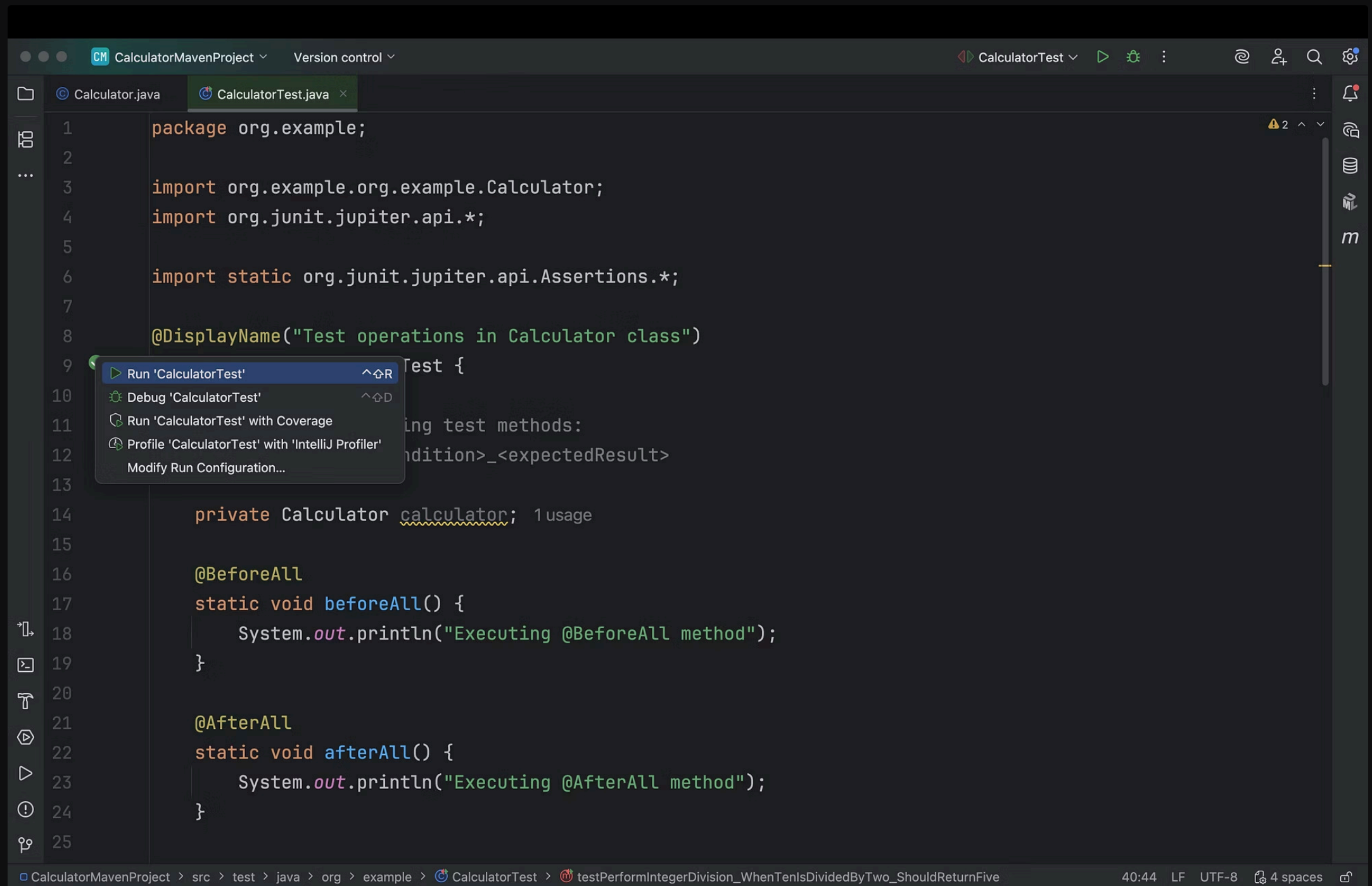


```
9      public class CalculatorTest {
38      @Test
39      public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
40          System.out.println("Test 10/2 = 5");
41          // Arrange (Given)
42          Calculator calculator = new Calculator();
43          int dividend = 10;
44          int divisor = 2;
45          int expectedResult = 5;
46
47          // Act (When)
48          int actualResult = calculator.performIntegerDivision(dividend, divisor);
49
50          // Assert (Then)
51          assertEquals(expectedResult, actualResult, message: "10/2 did not yield 5");
52      }
53
54      @DisplayName("Test 113-13=100")
55      @Test
56      public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturn0
57          System.out.println("Test 113-13=100");
58          Calculator calculator = new Calculator();
59          int minuend = 113;
60          int subtrahend = 13;
61          int expectedResult = 100;
```

Fourth step: Include `System.out.println` statements to make it easier to follow the output of the test execution (**Lines 40 and 57**)

Example: Applying Lifecycle Methods(contd.)

Run all test cases and follow the console output



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The file 'CalculatorTest.java' is open, displaying the following code:

```
1 package org.example;
2
3 import org.example.org.example.Calculator;
4 import org.junit.jupiter.api.*;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 @DisplayName("Test operations in Calculator class")
9 class CalculatorTest {
10     // ...
11     // ...
12     // ...
13
14     private Calculator calculator;
15
16     @BeforeAll
17     static void beforeAll() {
18         System.out.println("Executing @BeforeAll method");
19     }
20
21     @AfterAll
22     static void afterAll() {
23         System.out.println("Executing @AfterAll method");
24     }
25 }
```

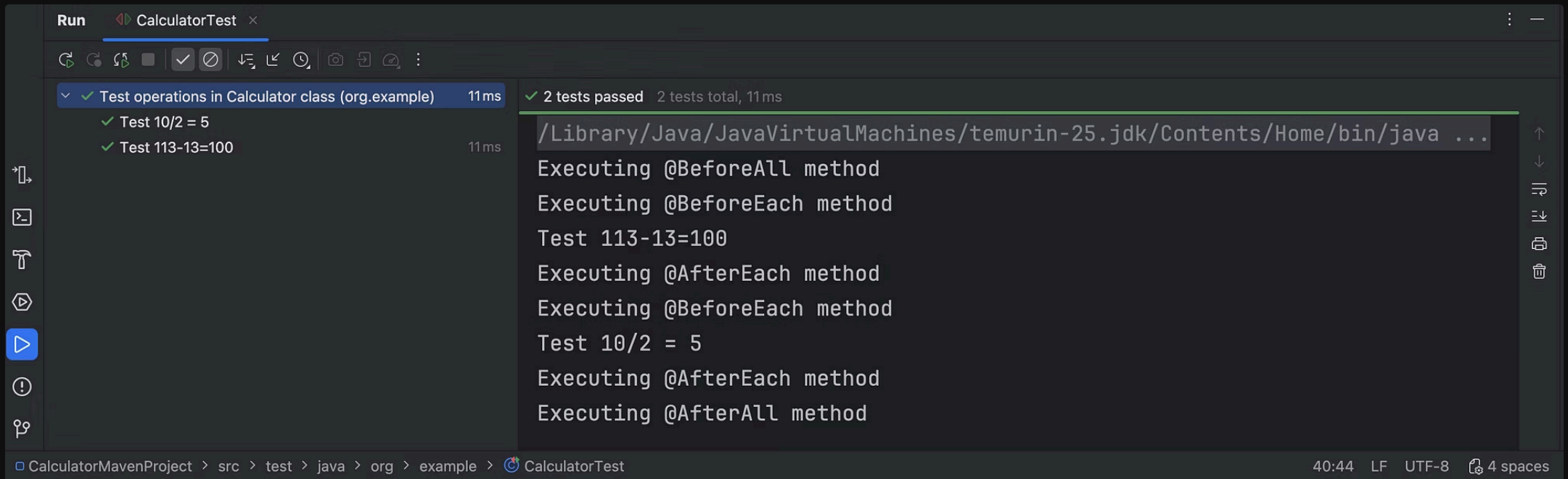
A context menu is open over the code, showing the following options:

- Run 'CalculatorTest' (⌘R)
- Debug 'CalculatorTest' (⌘D)
- Run 'CalculatorTest' with Coverage
- Profile 'CalculatorTest' with 'IntelliJ Profiler'
- Modify Run Configuration...

The bottom status bar shows the file path: 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive'. The status bar also displays '40:44', 'LF', 'UTF-8', and '4 spaces'.

Fifth step: Run all test cases. Can you predict the output without looking at the console?

Example: Applying Lifecycle Methods (Test results)



```
Run CalculatorTest x
[Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons]
Test operations in Calculator class (org.example) 11ms
  Test 10/2 = 5
  Test 113-13=100 11ms
2 tests passed 2 tests total, 11ms
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...
Executing @BeforeAll method
Executing @BeforeEach method
Test 113-13=100
Executing @AfterEach method
Executing @BeforeEach method
Test 10/2 = 5
Executing @AfterEach method
Executing @AfterAll method
CalculatorMavenProject > src > test > java > org > example > CalculatorTest 40:44 LF UTF-8 4 spaces
```

Exercise: Verify your understanding of this lesson by following each line of the output, contrasting it with the lifecycle annotations used in the CalculatorTest class!

Conclusion

- **Lifecycle annotations** provide a **clean**, maintainable **structure** for **tests**
- They **simplify setup** and **teardown** and **reduce repeated code**
- Correct use ensures **each test runs in isolation** with **predictable results**
- Essential for **scaling unit tests** in medium and large projects



Open Floor Discussion



Which lifecycle
hook feels most
useful for your
current project
structure?

Do you plan to
move object
initialization
into `@BeforeEach`?

Any shared
resources that
should be handled
in `@BeforeAll` or
`@AfterAll`?